

MID-STATE TECHNICAL COLLEGE • IT PROGRAMS

# Editors, Archives, and Compression

---

Unit 5 • Linux (10-151-105)

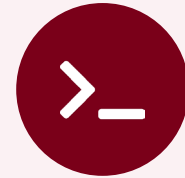


# Learning Outcomes



*By the end of this unit you will be able to:*

- **Edit configuration files from the command line using Vim and nano.**
  - Move between Vim's Command, Insert, and Execute modes with confidence.
  - Save, quit, and recover safely in both editors — with no GUI required.
- **Bundle, transfer, and restore files with tar and cpio.**
  - Compress data with gzip, xz, bzip2, zip, and 7z — and choose the right tool.
- **Verify file integrity with hashing, and understand HMAC.**
- **Use dd and ddrescue for block-level copying and disk recovery.**



## Two Halves

Edit the files — then  
protect and move them.

## LESSON 5.1

# Edit Text Files

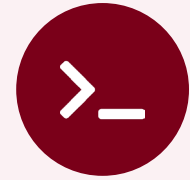
---



# Why You Edit Files at the Command Line



- **Linux keeps almost all configuration in plain text files — mostly under /etc.**
- Changing system behavior usually means editing one of those files and saving it.
- **Most servers have no graphical interface — a GUI is optional, and left out to stay small and secure.**
- So you edit from the keyboard, in a terminal editor. There is no settings dialog.
- **Being able to open, edit, save, and close a file in any terminal editor is non-negotiable.**



## No GUI

On a server, the  
terminal is the  
only editor you  
can count on.

# Configuration Files in Practice

---



- **Remote access over SSH is configured entirely in one text file:**
- `/etc/ssh/sshd_config` controls who may connect, whether root can log in remotely, password vs. key auth, and the login banner.
- **No dialog box exists — you open it in an editor, change directives, and restart the service.**
- Can't edit text from the terminal? Then you can't harden SSH — one of the most important things you do to a server.

```
# /etc/ssh/sshd_config
PermitRootLogin no
PasswordAuthentication no
Port 22

$ sudo systemctl restart sshd
```

# The Editors You'll Meet



| Editor       | What it is                                                                                           |
|--------------|------------------------------------------------------------------------------------------------------|
| <b>vi</b>    | The original Unix visual editor, later reimplemented as open-source software. Almost always present. |
| <b>vim</b>   | “Vi Improved” — the default, most capable terminal editor. The one to learn well.                    |
| <b>nano</b>  | A small, friendly terminal editor that uses Ctrl-key shortcuts instead of modes. Easiest to learn.   |
| <b>emacs</b> | A flexible, powerful, highly customizable editor with a devoted following.                           |
| <b>gvim</b>  | The graphical version of Vim, for systems that have a desktop.                                       |
| <b>gedit</b> | A simple GUI editor in the GNOME desktop — menu-driven and familiar.                                 |

***The two that matter on a server: vim and nano.***

# Vim: A Modal Editor



- **Installed nearly everywhere; extremely powerful, but unlike any editor you've used.**
- The key idea is modes: the same key does different things depending on the mode.
- Press `w` in one mode and you type “w”; in another, you save the file.
- **It feels strange at first, then becomes fast — your fingers rarely leave home row.**
- **When in doubt, press Esc — it always returns you to the safe home base, Command mode.**



## 3 Modes

Command, Insert,

and Execute —

learn how to

switch between them.

# Vim's Three Modes



*Command and Execute act on the file. Insert manages the text itself.*



# Command Mode: Move & Edit



| Key            | Action                                                       |
|----------------|--------------------------------------------------------------|
| <b>gg / G</b>  | Jump to the top / bottom of the file.                        |
| <b>42G</b>     | Jump to a specific line (line 42 here).                      |
| <b>h j k l</b> | Move the cursor left, down, up, right.                       |
| <b>d / dd</b>  | Delete the word at the cursor / the whole line.              |
| <b>3dd</b>     | Delete three lines at once (counts work with most commands). |
| <b>yy / p</b>  | Yank (copy) the current line / put (paste) it below.         |
| <b>ZZ</b>      | Write the file and quit Vim in one stroke.                   |

# Insert Mode: Entering Text



*Several ways to enter Insert mode — each starts typing in a different place:*

| Key      | Where typing begins                               |
|----------|---------------------------------------------------|
| <b>i</b> | Before the cursor (the standard, simplest entry). |
| <b>I</b> | At the beginning of the current line.             |
| <b>a</b> | After the cursor.                                 |
| <b>A</b> | At the end of the current line.                   |
| <b>o</b> | On a new line below the current one.              |
| <b>O</b> | On a new line above the current one.              |

***Press Esc when done to return to Command mode.***

# Execute Mode: Saving & Quitting



- From Command mode, press **:** to open a prompt at the bottom of the screen.
- The commands you'll use constantly are about writing the file and leaving.

| Command        | Action                                 |
|----------------|----------------------------------------|
| <b>:w</b>      | Write (save) the file.                 |
| <b>:w name</b> | Save under a new name.                 |
| <b>:q</b>      | Quit Vim.                              |
| <b>:wq</b>     | Write, then quit.                      |
| <b>:q!</b>     | Quit without saving — discard changes. |



## Escape Hatch

Made a mess?

Esc then :q!

leaves without

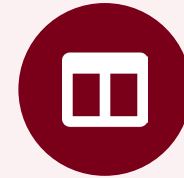
saving a thing.

# Split Windows in Vim



- **Divide the screen into windows — horizontal, vertical, or both.**
- Each window has Vim's full power; show different parts of one file, or two files.
- **Config files are the classic use: keep the comments at the top visible while you edit settings far below.**

| Keys                   | Result                         |
|------------------------|--------------------------------|
| <b>Ctrl+W</b> <b>s</b> | Split into horizontal windows. |
| <b>Ctrl+W</b> <b>v</b> | Split into vertical windows.   |
| <b>Ctrl+W</b> <b>w</b> | Cycle to the next window.      |



## Two at Once

Reference the  
docs up top while  
editing the  
settings below.

# nano: Simpler by Design



- **Less powerful than Vim, but far gentler — great for quick edits. (Spelled lowercase: nano.)**
- No modes. Open a file and start typing immediately.
- **Commands use the Ctrl key plus a letter; common ones show along the bottom of the screen.**
- In nano's help, Ctrl is written as a caret (^) — so ^X means Ctrl+X.

```
GNU nano  myfile.txt
Hello World!

^O Write Out    ^X Exit
^K Cut          ^U Paste
```

# nano Shortcuts



| Shortcut                      | Action                                   |
|-------------------------------|------------------------------------------|
| <b>Ctrl+A</b> / <b>Ctrl+E</b> | Move to the start / end of the line.     |
| <b>Ctrl+N</b> / <b>Ctrl+P</b> | Move down / up one line.                 |
| <b>Alt+\</b> / <b>Alt+/</b>   | Jump to the top / bottom of the file.    |
| <b>Ctrl+O</b>                 | Write out (save) the file.               |
| <b>Ctrl+K</b> / <b>Ctrl+U</b> | Cut the current line / paste (uncut) it. |
| <b>Ctrl+X</b>                 | Exit nano (prompts to save if needed).   |

*To copy a selection: mark with Ctrl+^, extend the highlight, copy with Alt+^, paste with Ctrl+U.*

# gedit and Other GUI Editors



- **gedit is the default graphical editor in the GNOME desktop — menu-driven, with syntax highlighting and spell-check.**
- Customizable through plugins; launch it from the CLI with the gedit command, same pattern as vim/nano.
- **But on the servers you'll administer, it usually isn't there — a GUI is exactly what a hardened server leaves out.**
- Fine when you have a desktop; the editor you can always count on lives in the terminal.



## When You Have It

Comfortable — but  
absent on most  
servers. Learn the  
terminal editors.

# Editors Across Rocky and Ubuntu



*Vim is Vim and nano is nano — the differences are small and practical:*

|                    | Rocky Linux 10 (Red Hat)           | Ubuntu Server 24.04 (Debian)        |
|--------------------|------------------------------------|-------------------------------------|
| Full Vim           | dnf install vim-enhanced           | apt install vim                     |
| nano               | Usually present (dnf install nano) | Installed by default                |
| Default for visudo | Usually vi                         | Usually nano                        |
| Change default     | Set \$EDITOR                       | Set \$EDITOR or update-alternatives |

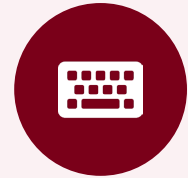
***Don't assume your editor is installed on a fresh server — knowing both means you're never stuck.***



# Hands-On: Editing a Config File



- **Back up first:** `cp /etc/ssh/sshd_config /root/sshd_config.old`
- Open it in Vim — navigate, change a directive, `:wq` to save.
- Open it again in nano — edit, `Ctrl+O` to save, `Ctrl+X` to exit.
- **Compare the experience: modal vs. modeless editing.**



## Same File

Edit `sshd_config`

in both editors

and feel the

difference.

**LIVE DEMO**

*Edit `/etc/ssh/sshd_config` in Vim, then in nano — backing up before each change.*

# Lesson 5.1 Recap

---



- **Linux configuration lives in text files — you edit them from the terminal.**
- **Vim is modal: Command (navigate/edit), Insert (type), Execute (save/quit).**
- Esc is home base; :wq saves and quits; :q! bails out without saving.
- **nano is modeless and shows its Ctrl-key shortcuts on screen.**
- GUI editors like gedit exist, but the terminal editors are what you can rely on.

## LESSON 5.2

# Manage Text Files

---



# Protecting and Moving Files



- **Now that you can edit files, the next job is to protect and move them.**
- Back up a config before you change it; bundle many logs into one archive; compress a backup so it fits; prove a file hasn't changed.
- **Archive with tar and cpio.**
- **Compress with gzip, xz, bzip2, zip, 7z.**
- **Verify with hashing (md5sum, sha256sum) and HMAC.**
- **Copy and rescue raw blocks with dd and ddrescue.**



**Bundle. Compress.**

Verify.

The backup

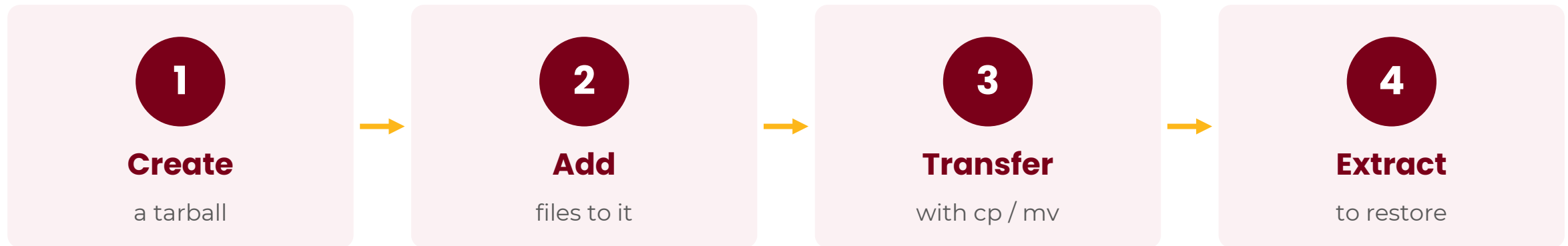
lifecycle in

three habits.

# Bundling Files with tar



- **Bundling many files into one makes them easier to back up, copy, and transfer.**
- The standard tool is tar (“tape archiver”) — flexible, efficient, can compress too.



```
tar {options} {tarball name} {files...}
```

# tar: Core Options



| Option    | What it does                                                |
|-----------|-------------------------------------------------------------|
| <b>-c</b> | Create a new tarball.                                       |
| <b>-x</b> | Extract files from a tarball.                               |
| <b>-t</b> | List (test) the contents without extracting.                |
| <b>-v</b> | Verbose — show each file as it is processed.                |
| <b>-r</b> | Append more files to an existing archive.                   |
| <b>-f</b> | Specify the tarball filename. Always required.              |
| <b>-p</b> | Preserve permissions and ownership — important for backups. |

# tar in Action



*With files file1–file4:*

```
● ●  
  
# Create (the backup step)  
$ tar -cvf filearchive.tar file1 file2 file3  
  
# List what is inside  
$ tar -tf filearchive.tar  
  
# Append another file  
$ tar -rf filearchive.tar file4  
  
# Extract (the restore step)  
$ tar -xf filearchive.tar
```

# cpio: Copy In, Copy Out



- **A close relative of tar, found on most Linux systems.**
- Its name describes its two directions — and it leans on redirection (> and <).
- **-o (copy out): files OUT of the filesystem into an archive — the backup direction.**
- **-i (copy in): files IN to the filesystem from an archive — the restore direction.**
- cpio can even produce tar-format files, but tar is what you reach for first.

```
# back up current dir
$ ls | cpio -ov \
    > /tmp/arch.cpio

# restore
$ cpio -iv \
    < /tmp/arch.cpio
```



# Why Compress?



- **Compression encodes data to use fewer bits — cheaper to store, faster to transfer.**
  - Lossless removes only redundant data (used for backups and files).
  - Lossy drops non-critical data (images, audio) — not for backups.
- **Compression ratio = uncompressed size ÷ compressed size.**
  - A 5 MB file that shrinks to 1 MB has a 5:1 ratio.
- **No tool wins on both speed and ratio — which is why several exist.**



## Speed vs. Ratio

Every compressor  
trades one  
against the  
other.

# The Compression Toolkit



Similar option style across all: *-d decompress, -v verbose, -t test integrity.*

| Tool  | Format | Character                                                                                  |
|-------|--------|--------------------------------------------------------------------------------------------|
| gzip  | .gz    | Fast, ubiquitous, modest ratio. Decompress with gunzip. Read in place: zcat, zless, zgrep. |
| xz    | .xz    | Best compression ratio of the group — at the cost of speed.                                |
| bzip2 | .bz2   | Middle of the pack on both. Decompress with bunzip2; read with bzipcat, bzless.            |

*zcat, zless, and zgrep mirror cat, less, and grep — reading .gz files without expanding them.*

# zip and 7z



- **Unlike gzip/xz/bzip2, these both ARCHIVE and COMPRESS in one step.**
- zip — produces the .zip files familiar from Windows; expand with unzip.
  - Options include -e (encrypt) and -r (recurse into directories).
- **7z (7-Zip) — cross-platform, open source, very high compression, great for large files.**
  - -mx sets the compression level from 1 (fast) to 9 (smallest).

```
$ zip -r out.zip docs/
$ unzip out.zip

$ 7z a -tzip \
    backup.zip *.txt -mx9
```

# Choosing a Compression Tool



| Tool         | Choose it when...                                                       |
|--------------|-------------------------------------------------------------------------|
| <b>gzip</b>  | Speed matters most, storage is plentiful — fastest, lowest ratio.       |
| <b>xz</b>    | Storage is at a premium and time is not — best ratio, slowest.          |
| <b>bzip2</b> | You want a balance, for data that is rarely decompressed.               |
| <b>7z</b>    | You are archiving very large files and want flexible, high compression. |

*Use unzip for .zip files, gunzip for .gz files.*

# Combining tar with Compression



- Usually you want to bundle AND compress in one move — tar calls the compressor for you.
- Add -z for gzip (.tar.gz), or -j for bzip2 (.tar.bz2).
- Use the same -z or -j alongside -x to extract and decompress in one step.

```
# bundle + gzip
$ tar -czvf my.tar.gz *.txt

# bundle + bzip2
$ tar -cjvf my.tar.bz2 *.txt

# extract gzip
$ tar -xzvf my.tar.gz
```

# Hands-On: Bundle, Compress, Extract



- **Bundle a directory of logs into a single .tar.gz with one command.**
- List the contents with `-tf` before extracting — a safe habit.
- **Extract into a clean directory and confirm permissions survived (-p).**
- Repeat with `bzip2` and compare the resulting file sizes.



## Compare Sizes

Same files,  
different tools —  
watch the ratio  
change.

### LIVE DEMO

`tar -czvf logs.tar.gz /var/log/demo → list → extract → verify.`

# Proving Files Haven't Changed



- **Data integrity = data stays accurate and unchanged in storage, use, and transit.**
- Integrity checking proves a file hasn't been altered — vital right after a backup.
- **The standard technique is hashing: an algorithm turns a file into a short, fixed fingerprint.**
- **Hash today, hash later, compare. Identical = unchanged. Different = something changed.**



## One Fingerprint

Any change —  
even one byte —  
produces a  
different hash.

# Weak vs. Strong Hashes



*Two attacks drive the distinction: collisions (two inputs, one hash) and short output lengths.*

| Strength                         | Algorithms                                                                                                      |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>Weak — avoid for security</b> | MD5 (128-bit) and SHA-1 (160-bit). OK for casual integrity checks, broken for security. SHA-1 retiring by 2030. |
| <b>Strong — prefer these</b>     | SHA-256, SHA-384, SHA-512, SHA-3, and (for passwords) Argon2.                                                   |

***Plan to remove weak hash algorithms from your workflows.***



# md5sum and sha#sum in Action



- **md5sum {file}** prints a file's MD5 fingerprint.
- sha1sum, sha256sum, sha384sum, sha512sum work identically with stronger algorithms.
- **Add -c to check files against a saved list of hashes.**

```
$ md5sum myfile.txt
8ddd8be4...  myfile.txt
$ echo more >> myfile.txt
$ md5sum myfile.txt
926c68c2...  myfile.txt
# different bytes =
# different hash
```

*A matching hash proves only that contents are unchanged — not that a file is authentic, safe, or confidential.*

# HMAC: Integrity + Authenticity



- A plain hash proves a file is unchanged — but not who sent it.
- **HMAC (Hash-based Message Authentication Code) adds a shared secret key to a hash function.**
- The recipient recomputes the HMAC with the same secret; a match confirms integrity AND origin.
- **Used under the hood by protocols like HTTPS and SMTP.**
- Advantage: checks integrity and authenticity together, without exchanging public/private keys.



## Hash + Secret

Proves the file is  
unchanged and  
from a known  
sender.

# dd: Block-Level Copying



- **Copies and converts raw blocks — below the level of individual files.**
- Named with operands: if= (input), of= (output), bs= (block size), count=, status=progress.
- **Clone a partition or capture a whole drive to an image and write it back.**

```
$ dd if=/dev/sda1 \  
    of=/dev/sdb2  
$ dd if=/dev/sda \  
    of=drive.iso
```



## CAUTION

Sometimes called “disk destroyer.” It writes raw blocks instantly and will overwrite a whole partition if you swap if= and of=. No confirmation, no undo — confirm device names with lsblk and read the command twice before you press Enter.

# ddrescue: Rescuing a Failing Disk



- **When a disk is failing rather than healthy, ddrescue is the right tool.**
- **Built for rescue: makes multiple passes, pulling off all readable data and working around bad sectors.**
- Syntax names the FAILING source first, the WORKING destination second.
- **Options: -r sets retry attempts on bad sectors, -v adds detail.**
- When someone hands you a drive that's "making noises," this is what you reach for.

```
$ ddrescue \  
    /dev/sda \  
    /dev/sdb \  
    -r3 -v
```

# Archive Tooling Across Rocky and Ubuntu



*tar, gzip, xz, and bzip2 are present and identical on both. A few helpers may need installing:*

| Tool                    | Rocky Linux 10 (Red Hat)               | Ubuntu Server 24.04 (Debian) |
|-------------------------|----------------------------------------|------------------------------|
| tar / gzip / xz / bzip2 | Present by default                     | Present by default           |
| zip / unzip             | dnf install zip unzip                  | apt install zip unzip        |
| 7-Zip                   | dnf install p7zip p7zip-plugins (EPEL) | apt install p7zip-full       |
| Command name            | 7z / 7za                               | 7z / 7za                     |

*The commands and options behave the same — the only divergence is which package provides them.*

# Security Focus: Safe Edits, Trustworthy Backups



*The two halves of this unit each carry a security habit worth building now.*

- **Back up before you edit** — copy a config to a .old name; one bad line in sshd\_config can lock you out.
- **Preserve permissions in archives** — use tar -p so ownership and mode bits survive the round trip.
- **Distrust untrusted archives** — inspect with tar -tf before extracting (path traversal risk).
- **Verify with a hash** — hash a backup after creating it; a mismatch means it changed or corrupted.
- **Retire weak hashes** — MD5 and SHA-1 are fine for casual checks, broken for security; prefer SHA-256+.

**Why it matters:** *a backup you can't trust is worse than none — it gives false confidence. Copy first, preserve permissions, verify with a hash.*

# Your Lab This Week

---



- **NETLAB+ Lab: Editors, Archives, and Compression — performed on Rocky and Ubuntu.**
- Back up a configuration file, then edit it in both Vim and nano.
- Bundle a directory with tar, compress it with more than one tool, and extract it again.
- **Verify a file's integrity with a hash before and after a change.**
- Graded on the observable end state — capture your evidence before the reservation ends.



## **Prove It**

Real tasks on  
both distros —  
from a known-good  
snapshot.

# Key Takeaways

---



- **Linux configuration lives in text — you edit it from the terminal with Vim or nano.**
- **Vim is modal (Command / Insert / Execute); Esc is home, :wq saves, :q! bails.**
- **tar bundles files; add -z or -j to compress; -p preserves permissions.**
- **Pick a compressor for your goal: gzip (speed), xz (ratio), bzip2 (balance), 7z (large files).**
- **Hash to prove integrity; prefer SHA-256+; HMAC adds authenticity with a shared key.**
- **dd copies raw blocks (handle with care); ddrescue recovers from failing disks.**



# Questions?

Unit 5 · Editors, Archives, and Compression

